Lab 5: Threads

1. Command:

```
ls
mkdir threads
cd threads
nano thread.c
```

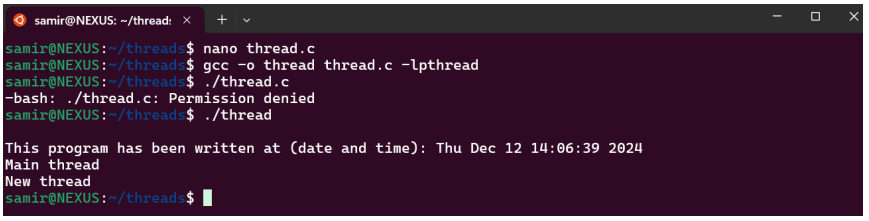
Code: `thread.c`

```
#include<stdio.h>
#include<pthread.h>
#include<time.h>

void * show(void * u){
    printf("New thread\n");
}

int main(){
    pthread_t tid;
    time_t t;
    time(&t);
    printf("\nThis program has been written at (date and time): %s", ctime(&t));
    pthread_create(&tid,NULL,&show,NULL);
    printf("Main thread\n");
    pthread_join(tid,NULL);
    return 0;
}
```

Compile and Run

```
gcc -o thread thread.c -lpthread
./thread
```

Output



Interpretation

Creating new thread using `pthread_create()` function and executing a user-defined function `show()`.

2. Command: nano thread_two.c

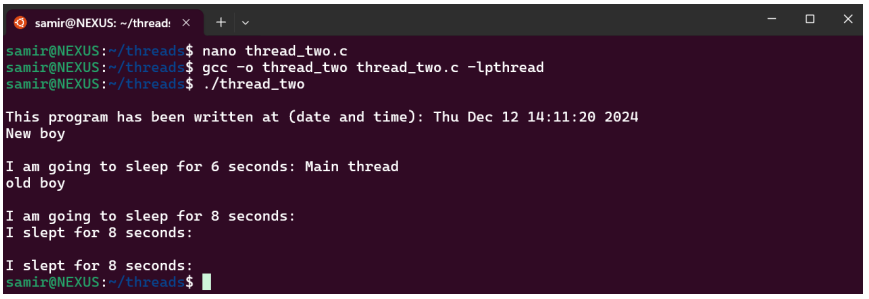
Code: `thread_two.c`

```c
#include<stdio.h>
#include<pthread.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>

int value=1;

void * sleep_a(void * u){
    printf("New boy\n");
    value=value+5;
    printf("\nI am going to sleep for %d seconds: ",value);
    sleep(value);
    printf("\nI slept for %d seconds: \n",value);
}

void * sleep_b(void *u){
    printf("old boy\n");
    value=value+2;
    printf("\nI am going to sleep for %d seconds: ",value);
    sleep(value);
    printf("\nI slept for %d seconds: \n",value);
}

int main(){
    pthread_t tid,tid2;
    time_t t;
    time(&t);
    printf("\nThis program has been written at (date and time): %s", ctime(&t));
    pthread_create(&tid,NULL,&sleep_a,NULL);
    pthread_create(&tid2,NULL,&sleep_b,NULL);
    printf("Main thread\n");
    pthread_join(tid,NULL);
    pthread_join(tid2,NULL);
    return 0;
}
```

Compile and Run

```
gcc -o thread_two thread_two.c -lpthread
./thread_two
```

Output



Interpretation

The `pthread_join()` function waits for the specified thread to terminate.

3. Command: nano thread_process.c

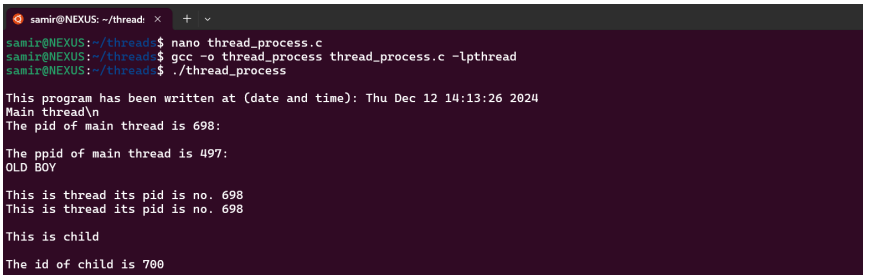
Code: `thread_process.c`

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#include<stdlib.h>
#include<time.h>

void * show(void * u){
    int pid;
    printf("OLD BOY\n");
    printf("\nThis is thread its pid is no. %d\n",getpid());
}

int main(){
    int pid;
    pthread_t tid;
    pthread_t tid_child;
    time_t t;
    time(&t);
    printf("\nThis program has been written at (date and time): %s", ctime(&t));
    pthread_create(&tid,NULL,&show,NULL);
    printf("Main thread\\n");
    printf("\nThe pid of main thread is %d: \n",getpid());
    printf("\nThe ppid of main thread is %d: \n",getppid());
    pid=fork();
    if(pid==0){
        printf("\nThis is child\n");
        printf("\nThe id of child is %d\n",getpid());
        printf("\nMy parent is %d: ",getppid());
        exit(0);
    }
    pthread_join(tid,NULL);
    return 0;
}
```

Compile and Run

```
gcc -o thread_process thread_process.c -lpthread
./thread_process
```

Output



Interpretation

Creating thread and its child thread.

4. Command: nano thread_argument.c

Code: `thread_argument.c`

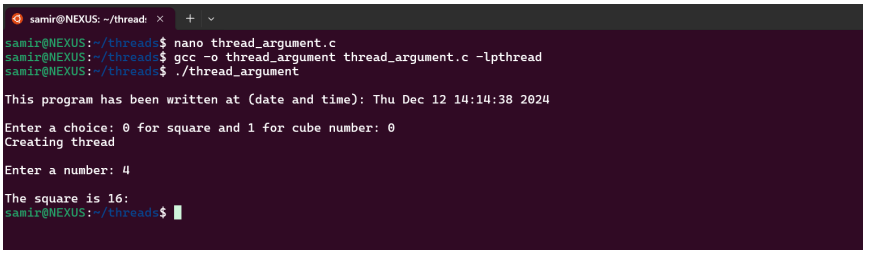
```
#include <pthread.h>
#include <stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<time.h>

int global;

void square(){
    global = global * global;
}

void cube(){
    global = global * global * global;
}

void *PrintHello(void *choice) {
    int *id_ptr, taskid;
    sleep(1);
    id_ptr = (int *) choice;
    taskid = *id_ptr;
    printf("\nEnter a number: ");
    scanf("%d", &global);
    if(taskid == 0){
        square();
        printf("\nThe square is %d: \n", global);
    }
    else if(taskid == 1){
        cube();
        printf("\nThe cube is %d: \n", global);
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t threads;
    int *choice = malloc(sizeof(int*));
    int rc;
    time_t t;
```

```
    time(&t);
    printf("\nThis program has been written at (date and time): %s", ctime(&t));
    printf("\nEnter a choice: 0 for square and 1 for cube number: ");
    scanf("%d", choice);
    printf("Creating thread \n");
    rc = pthread_create(&threads, NULL, PrintHello, (void *) choice);
    if (rc) {
        printf("ERROR; return code from pthread_create() is %d\n", rc);
        exit(-1);
    }
    pthread_exit(NULL);
    free(choice);
}
```

Compile and Run

```
gcc -o thread_argument thread_argument.c -lpthread
./thread_argument
```

Output



Interpretation

Demonstration of creating thread by passing function and its argument as parameter to the `pthread_create()` function.